

Java OOP Interview & Exam Preparation Guide

Complete Q&A Compilation

Java OOP Interview & Exam Preparation Guide

Java OOP Interview & Exam Preparation Guide

A complete question-and-answer compilation covering Object-Oriented Programming in Java, from fundamentals through advanced design.

1. What is Object-Oriented Programming? Why use it instead of procedural programming?

Definition: OOP is a programming paradigm based on the concept of “objects” — bundles of data (attributes) and behavior (methods) modeled on real-world entities.

Procedural vs OOP:

Feature	Procedural	OOP
Focus	Functions	Objects (data + behavior)
Data Security	Exposed/global	Encapsulated
Reusability	Low	High (inheritance)
Real-world mapping	Difficult	Natural
Maintenance	Hard at scale	Easier (modular)

The 4 Pillars: - **Encapsulation** – wrapping data + methods, hiding internals (e.g., a medicine capsule). - **Abstraction** – showing only essential features (e.g., driving a car without knowing engine internals). - **Inheritance** – a class acquiring another’s properties (e.g., Dog inherits from Animal). - **Polymorphism** – same method behaving differently per object (e.g., a person acts differently as employee vs parent).

Real-world example: A Library Management System — Book, Member, Library are classes; a specific book is an object; encapsulation protects issue status; inheritance lets Student/Teacher extend Member; polymorphism lets `calculateFine()` behave differently per member type; abstraction hides internal borrowing logic.

Interview summary: OOP models real-world entities as objects combining data and behavior, using encapsulation for security, inheritance for reuse, polymorphism for flexibility, and abstraction for simplicity — making large systems easier to build and maintain than procedural code.

2. What is a class and what is an object?

Class: A blueprint/template defining attributes and behaviors — no memory used until instantiated.

Object: A real, memory-occupying instance of a class with actual values.

```
class Car {
    String brand; String color; int speed;
    void startEngine() { System.out.println(brand + " started!"); }
}
Car car1 = new Car(); // object 1
Car car2 = new Car(); // object 2, independent of car1
```

Memory: Class metadata lives in the Method Area; objects live in the Heap; references (car1, car2) live on the Stack.

Real-world: Car class = blueprint; your friend's actual red Honda = an object.

3. What is the difference between a class and an instance?

"Instance" and "object" mean the same thing — an instance is simply the realized, memory-allocated version of a class. The word "instance" is used specifically when emphasizing the class-to-object relationship (e.g., "car1 is an *instance of* Car"), while "object" refers to the entity generally. Java's instanceof operator checks this relationship directly: car1 instanceof Car → true.

4. What is encapsulation? How do you achieve it in Java?

Definition: Bundling data and methods into a single unit (class) and restricting direct access to internal data ("data hiding").

How to achieve it (3 steps): 1. Make instance variables private. 2. Provide public getters for read access. 3. Provide public setters (with validation) for write access.

```
class BankAccount {
    private double balance;
    public double getBalance() { return balance; }
    public void deposit(double amt) { if (amt > 0) balance += amt; }
    public void withdraw(double amt) { if (amt > 0 && amt <=
balance) balance -= amt; }
}
```

Real-world: A medicine capsule — you can't touch the medicine directly, only consume it as the capsule allows. Similarly, an ATM hides internal banking logic; you interact only through defined operations.

Benefits: data hiding, security, loose coupling, maintainability, ability to create read-only fields (getter only, no setter).

5. Why should instance variables be declared private? What real-world problem does this prevent?

Making variables private prevents **uncontrolled, direct access** that could put an object into an invalid state (e.g., age = -25, balance = -999999). It forces all changes through validated methods.

Real-world problem prevented: Without private, any code anywhere could set a bank balance to a negative number or a password field with no encryption — corrupting data integrity and creating security holes. private + validated setters guarantee the object's state is *always* valid.

6. What are getters and setters? Why not just make all fields public?

Getters/setters are public methods providing controlled read/write access to private fields.

Why not public fields: No validation possible, no way to make a field read-only, breaks encapsulation, can't change internal representation later without breaking external code, no logging/side-effects possible.

```
class Weather {  
    private double temperatureInKelvin;  
    public double getTemperature() { return temperatureInKelvin -  
273.15; } // shows Celsius  
    public void setTemperature(double celsius) { temperatureInKelvin  
= celsius + 273.15; }  
}
```

Internal storage changed from Celsius to Kelvin — but external code calling getTemperature()/setTemperature() never needed to change.

7. What is a constructor? How is it different from a normal method?

Constructor: Special block, same name as class, no return type, called automatically once when an object is created via new, used to initialize state.

Feature	Constructor	Method
Name	Same as class	Any name
Return type	None	Required
Invocation	Automatic (via new)	Explicit call
Times called	Once per object	Many times
Inherited?	No	Yes

Types: default (no-arg), parameterized, copy constructor (manual in Java).

8. What is a default constructor? When does Java provide one automatically?

A **default constructor** is a no-arg constructor **automatically inserted by the compiler** only if the programmer writes **no constructor at all**. It sets fields to Java's default values (0, false, null).

The moment you write **any** constructor (even parameterized), Java stops providing the default — you must write your own no-arg version if still needed.

9. Can a class have multiple constructors? What is this called?

Yes — this is **Constructor Overloading**: multiple constructors differing in number, type, or order of parameters, providing multiple ways to initialize an object.

```
class Student {
    Student() { this("Unknown", 0); }
    Student(String name) { this(name, 18); }
    Student(String name, int age) { this.name=name; this.age=age; }
}
```

this() chains constructors to avoid duplicated logic (must be the first statement).

10. What is the this keyword used for? Give an example resolving ambiguity.

this refers to the current object. Main uses: resolving naming conflicts, calling another constructor (this()), calling current class methods, passing current object as an argument, returning current object (method chaining).

```
class Student {
    String name;
    Student(String name) {
        this.name = name; // this.name = instance var, name =
parameter
    }
}
```

Without this, name = name; would just assign the parameter to itself, leaving the instance variable null.

11. What is inheritance? Give a real-world example.

Inheritance: A mechanism where a subclass acquires fields/methods of a superclass, forming an "IS-A" relationship, implemented via extends.

```
class Employee { String name; double salary; void work(){...} }
class Manager extends Employee {
    int teamSize;
    void conductMeeting(){...} // new method
    @Override void work(){...} // overridden method
}
```

Manager inherits name, salary, work() but adds conductMeeting() and overrides work().

Types: Single, Multilevel, Hierarchical, Multiple (not supported for classes), Multiple via interfaces.

12. How do you implement inheritance in Java? Which keyword is used?

Use **extends** for class-to-class (or interface-to-interface) inheritance, and **implements** for a class fulfilling an interface. A class can extend only one class but implements multiple interfaces.

13. What is a superclass and a subclass?

Superclass (parent/base class) — the class being inherited from.

Subclass (child/derived class) — the class that inherits, using extends. The relationship is one-directional: a superclass has no knowledge of its subclasses. Object is the ultimate root superclass of every Java class.

14. Does Java support multiple inheritance with classes? Why or why not?

No — to avoid the **Diamond Problem** (ambiguity when two parent classes share a method signature). Java achieves safe multiple inheritance through **interfaces** (implements), since interface methods (pre-Java 8) had no body, so there was nothing to conflict over.

```
class C {} // class C extends A, B ✗ not allowed
class D implements X, Y {} // ✔ allowed
```

15. What is the super keyword used for?

super refers to the immediate superclass. Three uses: 1. super() — call parent's constructor (must be first statement). 2. super.method() — call parent's overridden method version. 3. super.field — access a field hidden/shadowed by the subclass.

If a subclass constructor doesn't call super() explicitly, Java inserts a call to the parent's no-arg constructor automatically.

16. Can a subclass access private members of its superclass? Why or why not?

No. private restricts visibility strictly to the declaring class, even from subclasses — this enforces true encapsulation. The private field still exists in memory as part of the inherited object, but cannot be referenced by name from the subclass; access must go through inherited public/protected getters/setters.

17. What is method overriding? Give a real-world scenario.

Overriding: A subclass provides its own implementation of a method already defined in its superclass (same signature) — enables runtime polymorphism.

```
class Employee { double calculateSalary(){ return baseSalary; } }
class Manager extends Employee {
    @Override double calculateSalary(){ return baseSalary + bonus; }
}
```

Real-world: “Make payment” executed differently by a cash counter vs online gateway vs auto-debit.

18. What are the rules for method overriding (return type, access modifier, exceptions)?

- Same method name & parameter list.
 - Return type: same or **covariant** (subtype).
 - Access modifier: same or **wider**, never narrower.
 - Checked exceptions: same, narrower, or none — never new/broader. Unchecked exceptions: unrestricted.
 - private, static, final methods and constructors **cannot** be overridden.
-

19. What is the @Override annotation and why is it good practice?

@Override tells the compiler you intend to override a parent/interface method. If the signature doesn’t actually match (e.g., a typo), the compiler throws an error immediately — instead of silently creating an unrelated new method (a dangerous, hard-to-detect bug). It also improves readability by clearly signaling intent.

20. What is constructor chaining? How does super() help?

Constructor chaining: one constructor calling another — this() within the same class, super() to the parent class — to reuse initialization logic. super() ensures the parent’s state is initialized **before** the subclass adds its own, and must be the constructor’s first statement. If omitted, Java inserts a call to the parent’s no-arg constructor automatically.

21. What is polymorphism? Explain with Shape.draw().

Polymorphism (“many forms”): the same method/object behaves differently depending on context.

```
class Shape { void draw(){...} }
```

```

class Circle extends Shape { @Override void draw(){
System.out.println("Circle"); } }
class Square extends Shape { @Override void draw(){
System.out.println("Square"); } }

Shape[] shapes = { new Circle(), new Square() };
for (Shape s : shapes) s.draw(); // correct version auto-called for
each

```

Two types: **Compile-time** (overloading) and **Runtime** (overriding).

22. Difference between compile-time and runtime polymorphism?

	Compile-time	Runtime
Achieved via	Overloading	Overriding
Also called	Static/early binding	Dynamic/late binding
Resolved	At compile time	At runtime (JVM)
Requires inheritance	No	Yes
Based on	Argument types	Actual object type

23. What is method overloading? Example with different parameter lists.

Overloading: multiple methods, same name, different parameter lists (number/type/order).

```

int add(int a, int b) { return a+b; }
double add(double a, double b) { return a+b; }
int add(int a, int b, int c) { return a+b+c; }

```

Cannot overload based on return type alone.

24. Can you overload methods by changing only the return type? Why or why not?

No. Java's method signature = name + parameter list; return type is **excluded**. Allowing return-type-only differences would make a call like `add(5,10)` ambiguous, since the caller isn't required to use the return value at all.

25. What is dynamic method dispatch?

The mechanism by which the JVM decides, **at runtime**, which overridden method to invoke, based on the **actual object type**, not the reference type. This is what implements runtime polymorphism. It applies **only** to overridden instance methods — NOT to fields or static methods, which are resolved via the reference type (static binding) — a common interview trap.

26. What is abstraction? How is it different from encapsulation?

Abstraction: hides implementation complexity, exposes only essential features (“what,” not “how”) — via abstract classes/interfaces.

Encapsulation: hides data, protects internal state via `private` + getters/setters.

One-liner distinction: “Abstraction hides complexity; Encapsulation hides data.”

27. What is an abstract class? Can you instantiate it?

An **abstract class** (`abstract` keyword) cannot be instantiated, and may mix abstract methods (no body) with concrete methods (full implementation). `new AbstractClass()` is a compile error — but a reference of the abstract type can point to a concrete subclass object. Abstract classes CAN have constructors (invoked via `super()`) and static methods.

28. What is an abstract method? Can an abstract class have non-abstract methods?

An **abstract method** has no body — only a signature ending in `;`. It can exist only in an abstract class or interface, and cannot be `private`, `static`, or `final`.

Yes, an abstract class can (and typically does) mix abstract and concrete methods — shared logic goes in concrete methods; behavior that varies per subclass goes in abstract methods.

29. What is an interface? How does it differ from an abstract class (pre-Java 8 vs Java 8+)?

Interface: a contract of abstract methods (+ constants) a class must fulfill via `implements`.

- **Pre-Java 8:** 100% abstract — only `public static final` fields and abstract methods, no bodies at all.
- **Java 8+:** interfaces can have default and static methods with bodies (introduced to add methods like `forEach()` to `Collection` without breaking millions of existing implementations). Java 9 added private interface methods (internal helpers only).

Still true even in Java 8+: interfaces cannot have instance variables or constructors — the core structural difference from abstract classes remains.

30. Can an interface have constructors? Why or why not?

No. A constructor initializes object state at creation time — but interfaces can never be instantiated (`new InterfaceName()` is illegal), and interfaces have no instance variables (only constants) to initialize. There is genuinely nothing for a constructor to do.

31. What are the four access modifiers in Java and their visibility scope?

Modifier	Same Class	Same Package	Subclass (diff pkg)	World
private	✓	✗	✗	✗
default	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

Top-level classes can only be public or default (not private/protected).

32. Difference between default and protected access?

Both allow same-package access. **protected** additionally allows access from **subclasses in different packages** — but only via inheritance (`this`), not by creating a direct object reference of the parent class and accessing its member from outside the package.

33. What is a static variable? Real-world example.

A **static variable** belongs to the class, not any object — one shared copy across all instances.

```
class BankAccount {  
    static double interestRate = 4.0;    // shared by every account  
}
```

Changing `BankAccount.interestRate` instantly reflects across all existing account objects — just like a bank changing its interest rate for every customer at once.

34. What is a static method? Why can't it access instance variables directly?

A **static method** belongs to the class and can be called without creating an object (`ClassName.method()`). It cannot access instance variables directly because a static method might be called **before any object exists** — so there's no specific object's data for it to reference. It can only access instance data if given an object reference explicitly as a parameter.

35. Difference between instance and static (class) variables?

	Instance Variable	Static Variable
Belongs to	Each object	The class
Copies	One per object	One total, shared
Memory	Heap (with object)	Method Area
Exists without object	No	Yes

Example: each Student's rollNumber (instance) vs the shared schoolName (static).

36. Difference between == and .equals()?

== compares **references** (memory addresses). .equals() compares **logical content** — the default Object.equals() behaves like == unless overridden (as String does). For custom classes, override .equals() to compare meaningful fields.

```
String s1 = new String("Hi"); String s2 = new String("Hi");
s1 == s2;           // false
s1.equals(s2);      // true
```

37. Why override equals() and hashCode() together? What breaks if you don't?

The contract: if equals() says two objects are equal, hashCode() **must** be equal too. Hash-based collections (HashMap, HashSet) use hashCode() first to locate the correct "bucket," then equals() to confirm a match. If only equals() is overridden, logically-equal objects can land in different buckets, causing contains()/get() to silently return false/null even for objects that should match — no compile error, just silent, hard-to-debug incorrect behavior.

38. What is toString() and why override it for debugging?

toString() returns a String representation of an object; default returns ClassName@HashCode (useless). Overriding it to show actual field values makes System.out.println(obj), logs, exception messages, and printed collections **immediately meaningful** for debugging, instead of a cryptic hash.

39. What is object composition? car HAS-A Engine example.

Composition: a class holds another class's object as a field — a "HAS-A" relationship (vs inheritance's "IS-A").

```
class Engine { void start(){...} }
class Car {
    Engine engine;           // Car HAS-A Engine
    void startCar(){ engine.start(); } // delegation
```

}

Favored over inheritance for flexibility and looser coupling (“favor composition over inheritance”).

40. Difference between “IS-A” and “HAS-A” relationships?

IS-A = inheritance (Manager extends Employee). **HAS-A** = composition (Car contains Engine). Test: if “X IS-A Y” makes sense, use inheritance; if only “X HAS-A Y” makes sense, use composition.

41. Why “favor composition over inheritance”? Scenario where inheritance breaks.

Inheritance creates rigid, tightly-coupled hierarchies that can break as requirements evolve. **Classic example — Square extends Rectangle:** mathematically valid, but if Rectangle allows setting width/height independently, forcing Square to keep both equal breaks expected behavior when used polymorphically — violating the **Liskov Substitution Principle**. Java’s own Stack extends Vector is a real mistake: Stack inherits Vector.add(index, element), breaking its LIFO contract. Composition avoids this by exposing only intended behavior.

42. What is a marker interface? Example (Serializable, Cloneable).

A **marker interface** has **no methods at all** — it exists purely to “tag” a class so the JVM/framework can check instanceof MarkerInterface and treat it specially. Serializable marks a class as eligible for serialization; Cloneable marks a class as safe to clone(). Without the marker, these operations throw NotSerializableException/CloneNotSupportedException at runtime.

43. What is the diamond problem? How does Java avoid it with interfaces?

Diamond Problem: ambiguity when a class inherits the same method from two sources with no clear resolution — named for the diamond-shaped hierarchy diagram. Java avoids it for classes by disallowing multiple class inheritance. With Java 8 default methods, the problem can resurface if two interfaces provide conflicting default implementations — Java **forces a compile-time error**, requiring the implementing class to explicitly resolve it (e.g., InterfaceName.super.method()), rather than silently guessing.

44. What are default and static methods in interfaces (Java 8+)? Why introduced?

Default methods: interface methods with a body (default keyword), inherited automatically, overridable. **Static methods:** utility methods belonging to the interface itself, called via `InterfaceName.method()`, not inherited.

Why introduced: Primarily to add new methods (like `forEach()`) to existing interfaces (like `Collection`) for the Stream API/lambda, **without breaking** millions of existing implementing classes — default methods provide automatic fallback behavior.

45. Can an interface extend multiple interfaces? Can a class implement multiple interfaces?

Yes to both. `interface C extends A, B {}` is valid (unlike class inheritance). `class X implements A, B, C {}` is also valid. Interfaces always use `extends` to inherit from other interfaces — never `implements`.

46. Design a Payment system (CreditCard, UPI, NetBanking). Which OOP concepts apply?

Use an **abstract class** `Payment` holding shared state (`amount`, `transactionId`) and concrete methods (`validateAmount()`, `printReceipt()`), with an abstract `processPayment()` overridden by `CreditCardPayment`, `UPIPayment`, `NetBankingPayment`.

Concepts: Abstraction (hides how each method processes), Encapsulation (protected fields), Inheritance (shared base), Runtime Polymorphism (a loop over `List<Payment>` calls the correct `processPayment()` per object via dynamic dispatch).

47. Design a Vehicle rental system using inheritance and polymorphism.

Abstract `Vehicle` class holds shared fields (`registrationNumber`, `baseRatePerDay`) and concrete methods (`checkAvailability()`, `bookVehicle()`), with abstract `calculateRentalCost(days)` overridden differently by `Car`, `Bike`, `Truck`. `bookVehicle()` internally calls `calculateRentalCost()` — dynamic dispatch selects the correct formula automatically. New vehicle types (e.g., `Bus`) can be added with zero changes to existing code (Open/Closed Principle).

48. Notification system (Email, SMS, Push) — how do interfaces help?

Since Email/SMS/Push share **no common state** but DO share a common capability (“must send a message”), a **Notification interface** with a `send()` method is ideal — not an abstract class. Benefits: polymorphic processing of mixed notification lists, easy extension (adding `WhatsAppNotification` requires no existing code changes), and

supports dependency injection (OrderService depends on the Notification interface, not concrete classes) for flexibility and testability.

49. How is OOP used in a real e-commerce Order and Cart system?

- **Abstraction:** Product (abstract) hides tax calculation differences; Payment (abstract) hides processing differences.
- **Inheritance:** Electronics/Clothing extends Product; CreditCardPayment/UPIPayment extends Payment.
- **Composition:** CartItem HAS-A Product; Cart HAS-A list of CartItems; Order HAS-A items + a Payment.
- **Encapsulation:** Cart's item list and Order's status are private, changed only via controlled methods.
- **Polymorphism:** order.placeOrder() calls payment.processPayment(), dynamically dispatched to the correct payment type.

Order takes a **snapshot** of cart items at checkout so later cart edits don't affect a placed order.

50. Model a Shape hierarchy (Circle, Rectangle, Triangle) for polymorphic area calculation.

Abstract Shape class with shared color/displayColor(), and an **abstract** calculateArea() (no meaningless default, since "area" has no universal formula). Each subclass overrides with its own formula. A single loop (for (Shape s : shapes) s.calculateArea()) works correctly for any shape via dynamic dispatch — adding a new shape (e.g., Pentagon) requires zero changes to existing processing code.

51. How do you prevent a method from being overridden? Real-world reason.

Declare it **final**. Attempting to override throws a compile-time error. **Real-world reason:** a bank's verifyPIN() method should be final so no subclass (e.g., SavingsAccount) can override it to bypass security verification — guaranteeing consistent, non-negotiable behavior across the entire hierarchy.

52. Significance of final for variables, methods, and classes?

Applied to	Effect
Variable	Cannot be reassigned once initialized (constant)
Method	Cannot be overridden by any subclass
Class	Cannot be extended/inherited at all

String is final in Java specifically to prevent security risks from subclassing something used so pervasively. For object references, final locks the reference, not the object's internal mutable fields.

53. Purpose of constructor overloading for initialization flexibility?

Lets an object be created with **whatever amount of information the caller actually has** — minimal, partial, or full — instead of forcing every caller through one rigid constructor demanding all data upfront (leading to awkward placeholder values like `null/0`). Lighter constructors typically delegate via `this()` to the most complete constructor, centralizing actual initialization logic in one place.

54. What is object cloning? Shallow vs deep copy?

Cloning: creating a duplicate object via `clone()` (requires `Cloneable`).

- **Shallow copy** (default `Object.clone()`): copies primitive fields by value, but reference-type fields only copy the **reference** — original and clone **share** the same nested object.
- **Deep copy:** manually clones nested reference-type fields too, so original and clone are **completely independent**.

```
@Override
public Object clone() throws CloneNotSupportedException {
    Employee cloned = (Employee) super.clone();
    cloned.address = (Address) this.address.clone(); // deep copy
    return cloned;
}
```

55. Why does default `clone()` do a shallow copy? When is this dangerous?

`Object.clone()` is a low-level **native** method that copies each field's raw memory value. For reference fields, the "raw value" IS the memory address — so it just duplicates the reference, not the object. A true default deep copy would risk infinite loops on circular references and unintentionally clone things that shouldn't be (like live connections). **Dangerous** whenever a class has **mutable** reference-type fields (custom objects, `ArrayList`, etc.) — modifying the shared nested object through either the original or the clone silently corrupts the other. Safe for immutable fields like `String`.

56. What is garbage collection? How does it relate to object lifecycle?

GC is the JVM's automatic process of reclaiming memory of objects no longer **reachable** from any active reference — eliminating manual `free()/delete`. Object lifecycle: Creation → In Use → Unreachable → Garbage → Collected. Java's GC detects even "islands of isolation" (mutually-referencing but externally-unreachable objects). `System.gc()` only *suggests* collection — it's never guaranteed to run immediately.

57. What is the purpose of `finalize()`, and why is it deprecated?

`finalize()` was meant to let an object perform cleanup just before GC reclaimed it. **Deprecated (Java 9+)** because: no guarantee it ever runs (JVM crash/abrupt exit skips it), unpredictable timing, significant performance overhead (multi-pass GC), objects can dangerously “resurrect” themselves, and exceptions thrown inside are silently swallowed. Modern replacement: **try-with-resources** + **AutoCloseable**, which guarantees deterministic, immediate cleanup.

58. How does exception handling relate to OOP (custom exception classes)?

Every exception is an **object**, part of a real inheritance hierarchy rooted at `Throwable`. Custom exceptions directly apply: - **Inheritance**: class `InsufficientFundsException` extends `Exception`. - **Encapsulation**: custom fields (e.g., `requestedAmount`, `availableBalance`) bundled inside the exception object. - **Polymorphism**: `catch (Exception e)` can catch any subclass via the common parent reference type.

59. How would you design a custom exception hierarchy for an application?

Structure: **Root** custom exception (`BankingException` extends `Exception`) → **category-level** exceptions (`AccountException`, `TransactionException`) → **specific leaf** exceptions (`InsufficientFundsException`, `DailyLimitExceededException`). Catch blocks must go from most-specific to least-specific. New exception types can be added later (e.g., `CurrencyMismatchException` extends `TransactionException`) without modifying existing catch (`TransactionException e`) code — Open/Closed Principle applied to exceptions.

60. Difference between checked and unchecked exceptions (class design)?

Purely a matter of **which class you extend**: - **Checked** → extends `Exception` directly (not `RuntimeException`). Compiler forces try-catch or throws. - **Unchecked** → extends `RuntimeException`. No compiler enforcement.

Design guideline: checked for normal, expected, recoverable business scenarios (e.g., `InsufficientFundsException`); unchecked for programming bugs/invalid internal state (e.g., `IllegalArgumentException` for a negative constructor argument).

61. Why catch specific exceptions rather than generic `Exception`?

Different failures need different responses; a generic catch (`Exception e`) treats everything identically and can **silently swallow genuine bugs** (like a `NullPointerException` caused by an actual defect), masking them with a misleading, unrelated error message. Specific catching also gives access to exception-specific fields/methods. Broad

catch (Exception e) is only appropriate as a last-resort safety net at an application's outermost boundary — and even then, it should fully log the exception, not silently suppress it.

62. Can constructors throw exceptions? Real-world use case.

Yes. A constructor's job is to bring an object into a *valid* state — if input makes that impossible, it should throw rather than silently create a broken object. If a constructor throws, the object is **never created** — no partially-initialized object exists.

```
BankAccount(String holder, double initialBalance) {  
    if (initialBalance < 500)  
        throw new IllegalArgumentException("Minimum initial deposit  
is ₹500.");  
    ...  
}
```

This “fails fast,” making bugs far easier to trace to their true origin.

End of compiled Q&A guide.